Developing a Class-Based Ride Sharing System

Sabin Ranabhat

An assignment submitted in partial fulfillment of the requirements for MSCS-632 as part of the degree Master of Science

In

Advanced Programming Languages

School of Computing and Information Sciences

University of the Cumberlands

MSCS-632: Advanced Programming Languages

Dr. Jay Thom

October 04, 2026

Fall 2026

1. Introduction

Object-oriented programming helps developers organize software by bundling related data and behavior into classes and objects (Sebesta, 2018). While the core ideas of OOP are consistent across computer science, different languages implement them in very different ways.

C++ is a statically typed, hybrid language that was designed to add object-oriented programming to C while keeping high performance and low-level control (Stroustrup, 2013). Smalltalk, on the other hand, is a dynamically typed, pure object-oriented language where everything is an object and all operations happen through message sending (Goldberg & Robson, 1983).

For this assignment, I implemented the same Ride Sharing System in both C++ and Smalltalk. The system models a base Ride class, specialized StandardRide and PremiumRide classes, a Driver class that tracks assigned rides, and a Rider class that tracks requested rides. This report explains how encapsulation, inheritance, and polymorphism work in each implementation.

2. Core OOP Principles in C++ and Smalltalk

2.1 Encapsulation

Encapsulation means keeping an object's internal state hidden from outside code and only allowing interaction through defined methods (Sebesta, 2018).

- **In C++:** C++ controls visibility using explicit keywords: private, protected, and public (Stroustrup, 2013). In cpp/ride_sharing.cpp, I declared variables like rideID, pickupLocation, dropoffLocation, and distance as private in the Ride class. This prevents outside functions from modifying them directly. To let subclasses access the distance for fare calculations, I added a protected method called getDistance(). In the Driver and Rider classes, the lists of rides (assignedRides and requestedRides) are also private. The only way to add a ride is by calling addRide() or requestRide(), ensuring the internal list cannot be altered or corrupted from the outside.
- **In Smalltalk:** Smalltalk handles encapsulation by default without needing keywords like private or public (Goldberg & Robson, 1983). In Smalltalk, all instance variables are automatically private to the object. In my Ride, Driver, and Rider classes, variables such as distance, assignedRides, and requestedRides cannot be accessed directly by any other object. The only way to read or update data is by sending messages to the object (such as driver addRide: aRide or rider getRiderInfo). This means encapsulation is built directly into the language structure rather than configured through compiler keywords.

2.2 Inheritance

Inheritance allows a new class (subclass) to inherit fields and methods from an existing class (superclass), promoting code reuse and establishing clear relationships between types (Sebesta, 2018).

- **In C++:** In C++, inheritance is declared at compile time using the ':' syntax. Both StandardRide and PremiumRide inherit publicly from Ride:

```
class StandardRide : public Ride {
public:
    StandardRide(int id, const string &pickup, const string &dropoff, double miles)
        : Ride(id, pickup, dropoff, miles) {}
    ...
};
```

The subclasses use an initializer list to call the parent Ride constructor. By inheriting from Ride, both subclasses automatically get common attributes like rideID and pickupLocation, and they reuse the shared string-building logic in rideDetails().

- **In Smalltalk:** Smalltalk uses a single-inheritance hierarchy where every class descends from Object (Black et al., 2009). To create the subclasses, I sent the subclass: message to Ride:

```
createClass value: (Smalltalk at: #Ride) value: #StandardRide value: #() value:
'RideSharing'.
createClass value: (Smalltalk at: #Ride) value: #PremiumRide value: #() value:
'RideSharing'.
```

StandardRide and PremiumRide automatically inherit the instance variables (rideID, distance, etc.) and methods of Ride. Subclasses can immediately use inherited methods like rideDetails while defining their own specific calculations for fare.

2.3 Polymorphism

Polymorphism allows different classes to respond to the same function call or message in their own specific way (Sebesta, 2018).

- **In C++:** C++ implements runtime polymorphism through virtual functions and virtual method tables (vtables) (Stroustrup, 2013). In the Ride base class, I defined fare() and rideType() as pure virtual functions:

```
virtual double fare() const = 0;
virtual string rideType() const = 0;
```

This makes Ride an abstract class that cannot be instantiated on its own. StandardRide overrides fare() to calculate $2.50 + ($1.50 * distance), while PremiumRide calculates $5.00 + ($2.75 * distance). In main(), I store both standard and premium rides inside a single vector of smart pointers (vector<shared_ptr<Ride>>). When iterating through this collection, the program uses the vtable at runtime to call the correct fare() method for each ride type, printing the right price for each ride automatically.

- **In Smalltalk:** Smalltalk uses dynamic message dispatch for polymorphism (Sebesta, 2018). Because Smalltalk is dynamically typed, there is no need for virtual keywords or type declarations. In the base Ride class, I marked the abstract methods using self subclassResponsibility:

```
fare
    self subclassResponsibility.
```

Both StandardRide and PremiumRide implement their own fare and rideType methods. When rides are placed into an OrderedCollection and iterated over, the Smalltalk virtual machine looks up the #fare message selector in the receiver's class at runtime. The same message produces the correct standard or premium rate dynamically without any compile-time type checking.

3. Program Execution and Output Verification

Both implementations were compiled, executed, and verified to produce matching output across all components.

Figure 1

C++ Ride Sharing System Execution and Output

```
sadhanakhadka@Mac cpp % ls
ride_sharing            ride_sharing.cpp
sadhanakhadka@Mac cpp %
sadhanakhadka@Mac cpp % ./ride_sharing
=== Ride Sharing System ===

Polymorphic Ride List:
Ride ID: 101 | Type: Standard | Pickup: RTP | Dropoff: Downtown Raleigh | Distance: 12.00 miles | Fare: $20.50
Ride ID: 102 | Type: Premium | Pickup: Cary | Dropoff: RDU Airport | Distance: 8.00 miles | Fare: $27.00
Ride ID: 103 | Type: Standard | Pickup: Morrisville | Dropoff: Raleigh Union Station | Distance: 10.00 miles | Fare: $17.50
Ride ID: 104 | Type: Premium | Pickup: Raleigh | Dropoff: Research Triangle Park | Distance: 15.00 miles | Fare: $46.25

Driver Information:
Driver ID: 501 | Name: Alex Johnson | Rating: 4.8 | Assigned Rides: 2

Rider Information:
Rider ID: 301 | Name: Sam Smith | Requested Rides: 2

Rider Ride History:
  - Ride ID: 101 | Type: Standard | Pickup: RTP | Dropoff: Downtown Raleigh | Distance: 12.00 miles | Fare: $20.50
  - Ride ID: 103 | Type: Standard | Pickup: Morrisville | Dropoff: Raleigh Union Station | Distance: 10.00 miles | Fare: $17.50

Program completed successfully.
sadhanakhadka@Mac cpp %
```

Note. Terminal execution of cpp/ride_sharing.cpp showing polymorphic ride listing with specialized fares, driver ride assignment, and rider request history.

Figure 2

Smalltalk (Pharo) Ride Sharing System Execution and Output

```
sadhanakhadka@Mac smalltalk % ls
ride_sharing.st run.st
sadhanakhadka@Mac smalltalk % /Applications/PharoLauncher.app/Contents/MacOS/Pharo --headless ../Pharo.image st ride_sharing.st --quit
=== Ride Sharing System ===

Polymorphic Ride List:
Ride ID: 101 | Type: Standard | Pickup: RTP | Dropoff: Downtown Raleigh | Distance: 12.0 miles | Fare: $20.5
Ride ID: 102 | Type: Premium | Pickup: Cary | Dropoff: RDU Airport | Distance: 8.0 miles | Fare: $27.0
Ride ID: 103 | Type: Standard | Pickup: Morrisville | Dropoff: Raleigh Union Station | Distance: 10.0 miles | Fare: $17.5
Ride ID: 104 | Type: Premium | Pickup: Raleigh | Dropoff: Research Triangle Park | Distance: 15.0 miles | Fare: $46.25

Driver Information:
Driver ID: 501 | Name: Alex Johnson | Rating: 4.8 | Assigned Rides: 2

Rider Information:
Rider ID: 301 | Name: Sam Smith | Requested Rides: 2

Rider Ride History:
  - Ride ID: 101 | Type: Standard | Pickup: RTP | Dropoff: Downtown Raleigh | Distance: 12.0 miles | Fare: $20.5
  - Ride ID: 103 | Type: Standard | Pickup: Morrisville | Dropoff: Raleigh Union Station | Distance: 10.0 miles | Fare: $17.5

Program completed successfully.
```

Note. Execution of smalltalk/ride_sharing.st in Pharo Smalltalk demonstrating dynamic dispatch of rideDetails, message sending, and matching output formatting.

4. Conclusion

Working through this project showed how two distinct programming languages can implement the same object-oriented design. C++ gives developers fine-grained control over access levels and memory while using static typing and vtables to ensure type safety and high performance. Smalltalk offers a simpler, more uniform model where encapsulation is automatic and dynamic message passing makes polymorphism natural and flexible. Both languages successfully implement encapsulation, inheritance, and polymorphism, each reflecting their respective design goals.

References

- Black, A. P., Ducasse, S., Nierstrasz, O., & Pollet, D. (2009). *Pharo by example*. Square Bracket Associates. https://books.pharo.org/pharo-by-example/
- Goldberg, A., & Robson, D. (1983). *Smalltalk-80: The language and its implementation*. Addison-Wesley.
- Sebesta, R. W. (2018). *Concepts of programming languages* (12th ed.). Pearson Education.
- Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley Professional.

Github Reference:

- https://github.com/sranabhat51358/MSCS632-Assignment5.git